

Adversarial Attacks

Robust Deep Learning (Seminar)

Bjarne Thal (446558) and Marius Wrobel (456734)

RWTH Aachen University, Germany
{bjarne.thal, marius.wrobel}@rwth-aachen.de

1 Introduction

Neural Networks ...

- what are they
- general robustness problem/
- descr. of adv. examples, ...? !Combine with early adv. history
- relevance
- structure of this report

[BRIEF neural network + backpropagation introduction?]

In deep learning, networks with many layers are used to solve tasks, such as image classification, in which we call the network the classifier.

2 Adversarial Attacks

2.1 Definition

Adversarial Attacks can be described as inputting *adversarial examples* into a machine learning system. [BRB17] These two terms were introduced by [Sze+13] and have since been used in different variations, however we here use the following definition:

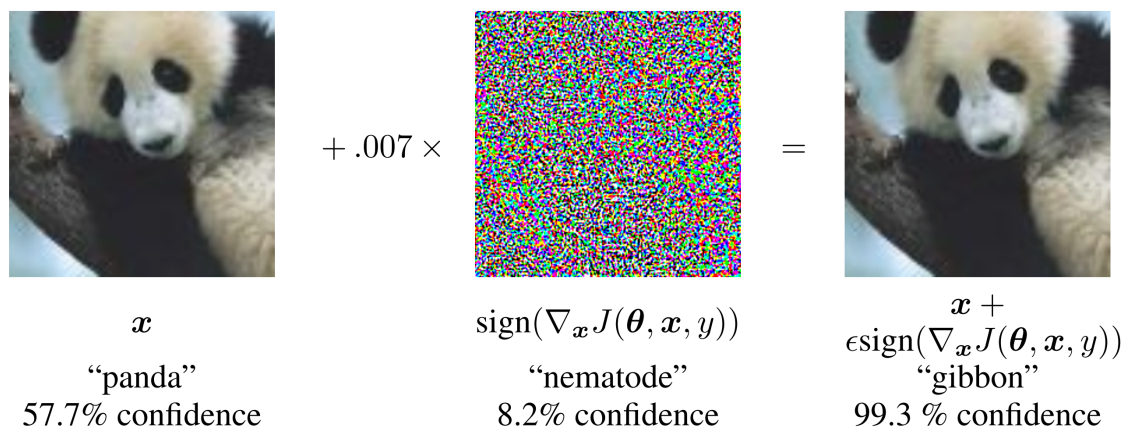


Figure 1: Classic illustration of an adversarial example. A small perturbation that is nearly imperceptible to humans is added to an image of a panda. While the original image is correctly classified as a panda with high confidence, the perturbed image is misclassified as a gibbon; also with high confidence. Adapted from [GSS14].

Definition 1 (Adversarial Example). An adversarial example is an input to a machine learning model that has been intentionally perturbed, typically in a subtle and often imperceptible manner, such that the model produces an incorrect prediction while the semantic content of the input remains unchanged.

24 *Machine Learning Models.* While adversarial examples are most commonly studied in the context of
 25 deep neural network classifiers,[Sze+13; Mad+17; Pit+19] the phenomenon is not limited to neural
 26 networks. Adversarial vulnerabilities have been demonstrated across a wide range of machine
 27 learning architectures, including decision trees, support vector machines, k -nearest neighbour
 28 classifiers, logistic regression models, ensemble methods and more. This suggests that susceptibility
 29 to adversarial manipulation is a general property of learned decision functions rather than a
 30 characteristic of a specific machine learning method.[Pap+16; Sze+13; GSS14]

31 *Intentional Perturbations.* The requirement that perturbations are *intentional* reflects the modern
 32 understanding of adversarial examples. The seminal work of Szegedy et al. [Sze+13] demonstrated
 33 the existence of inputs that caused misclassification despite appearing unchanged to humans,
 34 but did not initially frame these inputs primarily as the result of a deliberate attack. As the field
 35 evolved, researchers showed that such inputs can be generated systematically through optimization
 36 procedures and exploited by an adversary.[MFF16; Mad+17] Consequently, adversarial examples are
 37 now typically discussed in the context of adversarial attacks, where perturbations are deliberately
 38 crafted to induce erroneous model behaviour.[Cha+18; Pit+19; Cos+24]

39 *Subtle and Imperceptible Perturbations.* The notion of *subtle* perturbations can be interpreted in
 40 two related but distinct ways. First, adversarial perturbations are commonly constrained to be
 41 mathematically small according to a chosen distance metric. The most frequently used measures
 42 are the L_0 norm, which counts the number of modified features or pixels; the L_2 norm, which
 43 measures the Euclidean distance between the original and perturbed inputs; and the L_∞ norm,
 44 which limits the maximum change applied to any individual feature.[Cos+24] These norms define
 45 the perturbation budget available to an adversary and provide a mathematical notion of similarity
 46 between inputs. We will discuss these different norms in more detail in subsection 2.2.

47 Second, perturbations are often required to be imperceptible to humans. In the image domain,
 48 this means that the perturbed input should be visually indistinguishable or at least hardly distin-
 49 guishable from the original image, preserving its semantic meaning.[Mad+17; MFF16] Although we
 50 rarely see the perceptibility of the applied perturbations rigorously validated—for example, through
 51 experiments with human subjects or using LPIPS[Zha+18] as a proxy metric—it is nevertheless
 52 very common for authors to state, based on a visual assessment, that the adversarial examples they
 53 have generated are indistinguishable from the original to humans.[BRB17; Cha+18; Cos+24; Lia+22;
 54 Pap+16; PMG16]

55 *Confidence of Misclassification.* Some definitions additionally require adversarial examples to induce
 56 misclassifications with high confidence.[Mad+17] In practice, however, high-confidence errors are
 57 often a consequence of the optimization objective used during adversarial example generation rather
 58 than a defining characteristic of adversarial examples themselves. For example, early optimization-
 59 based attacks explicitly maximize the model loss or the probability assigned to a target class.[Sze+13]
 60 Consequently, contemporary literature generally treats high confidence as an attack objective rather
 61 than a necessary component of the definition.

62 2.2 Taxonomy

63 Adversarial attacks can be categorised according to several orthogonal dimensions, including
 64 the attacker’s knowledge of the target model, the stage at which the attack is performed, the at-
 65 tacker’s objective, and the constraint that bounds the perturbation. These dimensions are widely
 66 used throughout the literature and provide a useful framework for comparing attack methodolo-
 67 gies.[Pit+19; Cos+24] Figure 2 summarises these four dimensions together with their possible values;
 68 the attacks discussed in this survey are classified according to this taxonomy, with a complete
 69 overview provided in Table 1.

70 **By Attacker Knowledge** One of the most common distinctions is based on the amount of informa-
 71 tion available to the attacker about the target model.[Pit+19; Cos+24]

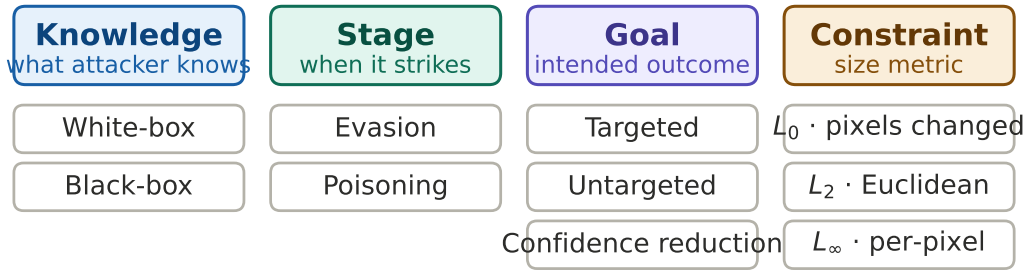


Figure 2: The four orthogonal dimensions along which adversarial attacks are classified: the attacker’s *knowledge* of the target model, the *stage* of the machine-learning pipeline at which the attack acts, the attacker’s *goal*, and the *constraint* (distance metric) that bounds the perturbation. A given attack is characterised by one choice from each column.

White-box Attacks. In a white-box setting, the attacker has complete knowledge of the target model, including its architecture, parameters, training procedure, and gradients. This level of access enables highly effective optimisation-based attacks that directly exploit gradient information to identify adversarial perturbations. Many influential attacks, including the Fast Gradient Sign Method (FGSM), Projected Gradient Descent (PGD), and the Carlini–Wagner attack, were originally proposed in this setting. White-box attacks are commonly used to evaluate the worst-case robustness of machine learning models.

Black-box Attacks. In a black-box setting, the attacker has no direct access to the internal structure of the model and can only observe its outputs in response to supplied inputs. Such attacks more closely resemble many real-world scenarios, where proprietary models are exposed only through prediction interfaces or APIs. Black-box attacks typically rely on repeated queries, surrogate models, or the transferability of adversarial examples between different models.

By Attack Stage Attacks can also be classified according to the stage of the machine learning pipeline at which they are carried out.[Cha+18; Pit+19]

Evasion Attacks. Evasion attacks are performed during the inference phase after a model has been trained. The attacker modifies input samples in order to induce misclassification while avoiding detection. Since they require no influence over the training process, evasion attacks are often considered the most practical and widely studied form of adversarial attack. Examples include adversarial images that cause image classifiers to produce incorrect predictions and manipulated network traffic designed to evade intrusion detection systems.

Poisoning Attacks. Poisoning attacks target the training phase by injecting maliciously crafted samples into the training dataset. The objective may be to degrade the model’s overall performance, manipulate its behaviour for specific inputs, or implant hidden backdoors that can later be triggered by an attacker. While poisoning attacks require some influence over the data collection or training pipeline, this assumption is realistic in settings that rely on crowdsourced, user-generated, or automatically collected data. Examples include recommendation systems trained on user feedback, spam filters trained on submitted emails, and machine learning models trained on large datasets scraped from the internet.

By Attack Goal A third dimension considers the attacker’s intended outcome.[Cha+18; Cos+24; BRB17]

102 *Targeted Attacks.* Targeted attacks seek to force the model to produce a specific attacker-chosen
 103 output. For example, an attacker may attempt to modify an image of a panda such that it is classified
 104 as a gibbon. Because the attack must drive the prediction toward a particular target class, targeted
 105 attacks are generally more difficult to perform than untargeted attacks.

106 *Untargeted Attacks.* Untargeted attacks aim only to cause a prediction error, but do not specify
 107 the resulting incorrect class. The attack is therefore successful as soon as the model’s prediction
 108 differs from the correct label. The attacker might aim at having the classifier recognize the image as
 109 showing anything but a panda.

110 *Confidence Reduction Attacks.* Rather than inducing a specific misclassification, confidence reduction
 111 attacks aim to decrease the model’s confidence in its predictions. Such attacks may not necessarily
 112 change the predicted class but instead increase uncertainty or reduce the probability assigned to the
 113 correct label. These attacks are particularly relevant in safety-critical applications, where confidence
 114 estimates are used to support downstream decision-making. By lowering confidence, an attacker
 115 may trigger fallback mechanisms, human intervention, or rejection of otherwise valid inputs.

116 **By Perturbation Constraint** Adversarial attacks are commonly categorised according to the distance
 117 metric used to constrain the perturbation applied to an input.[Cos+24; Lia+22] These constraints
 118 define what constitutes a *small* modification and therefore determine the attacker’s search
 119 space.[CW17] The majority of attacks in the literature operate under one of three norm-based threat
 120 models. Figure 3 illustrates these three norms geometrically.

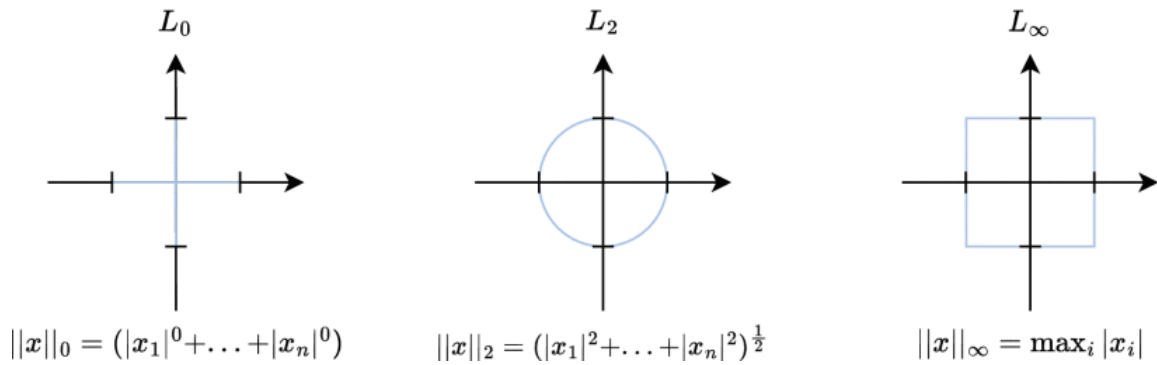


Figure 3: Geometric interpretation of the three norm constraints commonly used in adversarial machine learning. The unit balls of the L_0 , L_2 , and L_∞ norms are shown in a two-dimensional feature space. The L_0 norm constrains the number of modified features, the L_2 norm constrains the Euclidean distance between the original and perturbed input, and the L_∞ norm constrains the maximum change applied to any individual feature. Figure adapted from [Cos+24].

121 *L_0 -Constrained Attacks.* The L_0 norm measures the number of features that have been modified. In
 122 the context of image classification, this corresponds to the number of altered pixels regardless of the
 123 magnitude of the changes. Attacks operating under an L_0 constraint aim to minimise the number of
 124 modified features while still causing misclassification.[Cos+24; Lia+22] Examples for such attacks
 125 include Deepfool [MFF16] and Carlini & Wagner.[CW17]

126 *L_2 -Constrained Attacks.* The L_2 norm measures the Euclidean distance between the original and
 127 perturbed input vectors. Under this constraint, perturbations may be distributed across many
 128 features as long as the overall magnitude remains small.[Cos+24; Lia+22] The seminal work of
 129 Szegedy et al. [Sze+13] formulated adversarial example generation as an optimisation problem
 130 under an L_2 constraint. Due to its intuitive geometric interpretation, the L_2 norm remains widely
 131 used when evaluating adversarial robustness.[Cos+24]

Table 1: Overview of the attacks discussed in this survey, classified along the four dimensions of the taxonomy in Section 2.2. “Both” marks attacks with widely used targeted and untargeted variants; “–” marks poisoning attacks for which an input-space norm budget does not directly apply.

Attack	Knowledge	Stage	Goal	Constraint
L-BFGS [Sze+13]	White-box	Evasion	Targeted	L_2
FGSM [GSS14]	White-box	Evasion	Untargeted	L_∞
BIM / iterative [KGB17a]	White-box	Evasion	Both	L_∞
PGD [Mad+17]	White-box	Evasion	Untargeted	L_∞
DeepFool [MFF16]	White-box	Evasion	Untargeted	L_2
Carlini–Wagner [CW17]	White-box	Evasion	Targeted	$L_0/L_2/L_\infty$
ZOO [Che+17a]	Black-box	Evasion	Both	L_2
BadNets [GDG17]	White-box	Poisoning	Targeted	–
Targeted backdoor [Che+17b]	Black-box	Poisoning	Targeted	–
Poison Frogs [Sha+18]	White-box	Poisoning	Targeted	–
Clean-label backdoor [TTM19; SSP20]	White-box	Poisoning	Targeted	–
Narcissus [Zen+23]	Black-box	Poisoning	Targeted	L_∞

L_∞ -Constrained Attacks. The L_∞ norm limits the maximum change that can be applied to any individual feature.[Cos+24; Lia+22] In image domains, this ensures that every pixel is modified by at most a small amount. Many influential attacks, including FGSM and PGD, operate under an L_∞ constraint because it provides a straightforward approximation of perceptual similarity while enabling efficient optimisation.[GSS14; Mad+17]

Why do adv. examples exist? (Draft!) When adversarial examples were first empirically documented in 2013 [cite], it was first hypothesized that they were caused by a supposedly highly non-linear nature of neural networks. In 2014, Goodfellow et al.[cite] argued that they are instead a result of overly linear behaviour.

They considered a simple linear regression model [formula] and L_∞ perturbations. [Formula: separate x, perturbation] In high dimensions, if we choose perturbation=..., [term] can get large enough to impact the model prediction. By linearizing the loss function[see section below], they were able to construct adversarial examples on neural networks as well. On the contrary, models like RBF did not seem as vulnerable to adversarial examples, further giving support to their theory.

More detailed papers like [Adversarial examples are not bugs, they are features] -> Is it even possible to have networks not vulnerable to adversarial examples?

2.3 Notable Evasion Attacks

This section presents several notable adversarial attacks that have significantly influenced the development of adversarial machine learning. An overview of these attacks, together with their classification according to the taxonomy introduced in Section 2.2, is provided in Table 1.

L-BFGS The first widely recognised adversarial attack was introduced by Szegedy et al. [Sze+13]. Their work did not only show that machine learning models are vulnerable to inputs that lead to incorrect predictions while appearing unchanged to human observers, first describing the phenomenon of an adversarial example. They further introduced a fairly reliable method to find such examples.

To generate adversarial examples, the authors formulated the problem as a constrained optimisation task. Given an input x and a target class l , the objective is to find the smallest perturbation r such that the perturbed input $x + r$ is classified as the target class:

$$\min_r \|r\|_2 \quad \text{subject to} \quad f(x + r) = l, \quad (1)$$

where f denotes the classifier and $\|r\|_2$ represents the Euclidean distance between the original and perturbed input. Directly solving this problem is difficult because the classification constraint

163 $f(x + r) = l$ is highly non-linear and non-convex. To make the problem tractable, Szegedy et al.
 164 reformulated it as a continuous optimisation problem by replacing the hard classification constraint
 165 with a differentiable loss term that encourages the model to predict the target class. Specifically, they
 166 minimise

$$167 \quad \min_r c \|r\|_2 + \text{loss}_f(x + r, l), \quad (2)$$

168 subject to the box constraint

$$169 \quad x + r \in [0, 1]^m, \quad (3)$$

170 where $\text{loss}_f(x + r, l)$ measures how strongly the network predicts class l , $c > 0$ controls the
 171 trade-off between achieving the target misclassification and keeping the perturbation small, and the
 172 box constraint ensures that the perturbed input remains a valid image. This reformulation yields
 173 a differentiable objective whose gradients can be computed via backpropagation, allowing the
 174 perturbation to be found efficiently using the box-constrained Limited-memory Broyden–Fletcher–
 175 Goldfarb–Shanno (L-BFGS) optimisation algorithm. A line search over c is then performed to identify
 176 the smallest successful perturbation that causes the classifier to output the target class.

177 Although the L-BFGS attack is still computationally expensive compared to later methods, its
 178 significance lies in establishing the existence of adversarial examples and motivating the extensive
 179 research that followed. For this reason, it is widely regarded as the starting point of adversarial
 180 machine learning research.[GSS14; Cos+24; Lia+22]

181 **Fast Gradient Sign Method (FGSM) (Draft!)** Goodfellow et al hypothesized that the reason
 182 adversarial examples exist is that commonly, models are designed to behave mostly linear [GSS14].
 183 Previously, it was suspected that models were too non-linear [cite].

184 Based on this hypothesis, they came up with the fast gradient sign method to find L_∞ -constricted
 185 adversarial examples. L_∞ is chosen because it models sensor inaccuracies: a small perturbation in a
 186 sensor value would still be encoded to the same digital value. Unlike L-BGFS, they did not choose
 187 a specific target class they want the adversarial example to be mistaken for. Instead, they aim to
 188 maximize the loss value of the adversarial example. That means that we want to solve

$$189 \quad \arg \max_{x' \text{ s.t. } \|x' - x\|_\infty \leq \delta} L(\theta, x', y) = x + \arg \max_{\|\eta\|_\infty \leq \delta} L(\theta, x + \eta, y)$$

190 given a data point x , a limit δ , and a loss function $L(\theta, x, y)$, to find the ‘worst’ adversarial
 191 example (maximizes loss) in the L_∞ -ball around x

192 To do that, Goodfellow et al. linearize the loss function around x , giving us

$$193 \quad \arg \max_{\|\eta\|_\infty \leq \delta} L(\theta, x + \eta, y) = \arg \max_{\|\eta\|_\infty \leq \delta} L(\theta, x, y) + \nabla_x L(\theta, x, y)^T \eta = \arg \max_{\|\eta\|_\infty \leq \delta} \nabla_x L(\theta, x, y)^T \eta$$

194 We can easily see that $\eta = \delta \cdot \text{sign}(\nabla_x L(\theta, x, y))$ is a valid maximizer of the scalar product $\nabla_x L(\theta, x, y)^T \eta$.
 195 This corresponds to a single, as large as possible step in the direction of the gradient.

196 Since this approach only needs one backpropagation step instead of iterations like L-BGFS
 197 algorithm run, this was called the Fast Gradient Sign Method. It made adversarial training feasible,
 198 which aims to improve a model’s robustness by inserting adversarial examples into the training set.

199 **Variants of FGSM** Different variants of FGSM were proposed. [KGB17a], later [KGB17b].

200 The so-called **basic iterative method** applies FGSM multiple times with a small step size
 201 [KGB17a]. The adversarial example is given by

$$202 \quad X_0^{adv} = X, \quad X_{N+1}^{adv} = \text{Clip}_{X, \epsilon}(X_N^{adv} + \alpha \text{sign}(\nabla_X L(X_N^{adv}, y_{true})))$$

203 where $\text{Clip}_{X, \epsilon}$ implements pixel-wise constriction to the ϵ -neighborhood of X [KGB17a].

204 To find adversarial examples of a specific class y_{target} instead of maximizing the loss function,
 205 FGSM can be modified to maximize that classes probability $p(y_{target} | X)$. This can be achieved with a

single step in direction of $\text{sign}(\nabla_x \log(p(y_{\text{target}}|X)))$ [KGB17a], analogous to basic FGSM, or using multiple, like in the aforementioned basic iterative method [KGB17b]. A popular choice for target class is the one originally deemed least likely by the target model.

PGD Searching adversarial examples by using projected gradient descent (PGD) as proposed by Madry et al. [Mad+17] is similar to the basic iterative method. Both use first-order information, i.e. the gradient of the loss function, to iteratively maximize the loss on an input in the neighborhood of the real image. Instead of using multiple FGSM steps, though, Madry et al. use projected gradient descent directly.

They formulate the adversary's goal as....

$$\max_{\eta \in S} L(\theta, x + \eta, y)$$

Note the similarities to FGSM, by using $S = \{\eta \mid \|\eta\| \leq \delta\}$, we obtain its maximization problem. FGSM is therefore a specific method to tackle this general formulation.

As a generalized, stronger attack, they propose using PGD. They support their argument with an experiment, starting PGD starting from random points in the vicinity of x . While many different local maxima are found, they have roughly the same loss value. If an even better maximum existed, the argue it would be hard to find with any first-order method. Therefore, no other first-order method could find it in reasonable time either, making PGD a useful "universal" first-order adversary.

However, this strength has a caveat. As we will see in later sections 2.3, not every adversarial example corresponding to a local minimum is equally transferable to other models.

DeepFool DeepFool was introduced by Moosavi-Dezfooli et al. [MFF16] in 2016. The central idea of this iterative is to characterize adversarial vulnerability geometrically: rather than loosely optimizing an attack loss, DeepFool estimates the distance of an input to the closest decision boundary. For affine classifiers, this distance could be calculated in a closed form by projecting the input point onto the decision hyperplane. The highly nonlinear decision boundaries of deep neural networks are addressed by DeepFool by exploiting the observation that although the true boundary of a neural network may be highly curved, sufficiently small neighborhoods can often be approximated accurately by a hyperplane; the shortest path to this approximate boundary is assumed to provide a good estimate of the minimal adversarial perturbation. This auxiliary hyperplane is recomputed at each iteration from a first-order Taylor expansion of the classifier around the current point; as the point is repeatedly displaced onto the latest hyperplane, it moves closer and closer to the true, nonlinear decision boundary until it eventually crosses it. In mathematical terms, at an intermediate point x_i , the classifier is iteratively replaced by its local linear approximation,

$$f(x) \approx f(x_i) + \nabla f(x_i)^T (x - x_i), \quad (4)$$

which defines a locally linear decision boundary. The perturbation is obtained by projecting the current point onto this locally estimated boundary, yielding an update

$$x_{i+1} = x_i + r_i, \quad (5)$$

where r_i is the smallest perturbation that crosses the linearized boundary. The procedure is repeated until the classifier changes its prediction.

For multiclass classifiers, DeepFool extends this idea by considering the set of pairwise decision boundaries between the current class and competing classes. At each iteration, the method computes the locally closest boundary under the linear approximation and updates the input toward that boundary. Figure 4 visualises this iterative step.

DeepFool is made several important contributions to the field of adversarial attacks, the first major one being that it provided a practical approximation of minimal adversarial perturbations, making adversarial robustness palpable in terms of distance to the decision boundary. Secondly, it introduced a geometric perspective on adversarial attacks: instead of treating adversarial generation as a generic constrained optimization problem, it directly exploits the local structure of the classifier. Third, this formulation leads to substantially improved efficiency compared with previous

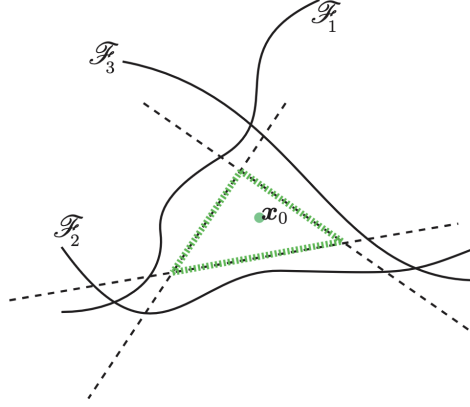


Figure 4: The approach of DeepFool estimates the position of the decision boundaries with the other classes with a linear representation using first-order Taylor expansion. By iteratively projecting the input point onto this auxiliary hyperplane, the perturbed point scooches closer and closer to the actual decision boundary of the model until eventually crossing it.

254 optimization-based attacks such as the L-BFGS attack of Szegedy et al. [Sze+13], which required solv-
 255 ing a computationally expensive optimization problem for each input. By using gradient information
 256 to directly estimate the boundary geometry, DeepFool achieves smaller perturbations than earlier
 257 fast gradient-based methods while requiring considerably less computation than general-purpose
 258 optimization approaches.

259 **Carlini-Wagner** The Carlini-Wagner (C&W) attack is an influential attack presented by Carlini
 260 et al. [CW17]. The method is building directly upon the original L-BFGS method by Szegedy et
 261 al. [Sze+13] by substantially improving the objective function of the optimization problem. They
 262 further introduced a trick to formulate the problem as an unconstrained optimization task, instead
 263 of a constrained one.

264 Just like L-BFGS, given an input example x and a target class t , the attack seeks an adversarial
 265 example $x' = x + \delta$ by solving:

$$266 \min_{\delta} \|\delta\|_p + c \cdot f(x + \delta), \quad (6)$$

267 of a logit-based loss function rather than optimizing the softmax output probabilities. The loss is
 268 defined as:

$$269 f(x') = \max \left(\max_{i \neq t} Z_i(x') - Z_t(x'), -k \right), \quad (7)$$

270 where $Z_i(x')$ denotes the logit corresponding to class i , and k is a confidence parameter control-
 271 ling the margin by which the target class must exceed competing classes. This formulation directly
 272 optimizes against the decision boundary of the classifier and thus efficiently calculates adversarial
 273 examples.

274 To address the pixel-range constraint of image inputs, the C&W attack introduces a clever
 275 variable transformation:

$$276 x' = \frac{1}{2}(\tanh(w) + 1), \quad (8)$$

277 where w is an unconstrained latent space optimization variable. This transformation converts
 278 the original constrained optimization problem into an unconstrained one while guaranteeing that
 279 the generated adversarial example remains within the valid input range. The attack then optimizes
 280 w using gradient-based optimization methods, typically the Adam optimizer[KB14].

The original work introduced variants based on different distance metrics, including L_0 , L_2 , and L_∞ norms, allowing the attack to evaluate robustness under different threat models.

The effectiveness of the C&W attack also demonstrated weaknesses in several proposed defenses. In particular, it was shown to successfully bypass defense distillation [Pap+16], a technique proposed to improve adversarial robustness by training a model using softened output probabilities from a teacher network. Although defense distillation appeared to increase resistance against earlier attacks, Carlini and Wagner demonstrated that the defense distillation method did in fact only obscure the model's gradient albeit not truly hiding it. By directly targetting the logit with their loss function, Carlini et al. were able to efficiently generate adversarial examples in spite of defense distillation by directly approximating the obscured decision boundary.

Transfer Attacks In a transfer attack, an adversary first obtains a substitute model that he has full access to as a stand-in for a target black-box model. On this substitute, white-box attacks can be performed to find adversarial examples. Since they generally transfer from one model to another, these can then be used to attack the actual target model.

Jacobian-based dataset augmentation Papernot et al. [Pap+17] introduced black-box attacks via transfer attacks where neither target model parameters nor a suitable data set is accessible to the attacker. Prior work by Biggio et al [Big+13] had introduced transfer attacks already, but granted access to such a dataset.

First, the substitute architecture is defined based on input and output of the target, e.g. one adapted to images. The architecture is overall not a deciding factor in the success of this approach.

The dataset restriction poses a difficulty for training the substitute. Papernot et al. solve this by querying the target on artificial data points to create a substitute dataset. Here, the selection of good data points to learn similar decision boundaries is crucial: a bad crafting strategy would require infeasible and highly suspicious amounts of queries to the target model. In particular, generation based on random noise does not work.

Instead, they rely on a heuristic to find out in which input direction the (substitute!) model's output changes and adding new samples focused on these directions. For that, they rely on the substitute model's Jacobian matrix (multidimensional derivative, each entry is one output coordinate partially derived by an input coordinate) at input points.

They start by collecting a set of samples somewhat representative of the target's input domain, such as different natural images. Next, an architecture is selected.

The following steps are then performed repeatedly: The adversary makes the target model label his current selection of samples, and then trains his substitute with the new examples. Then, he augments his dataset with additional input points.

Let $J_f(x)$ be the Jacobian of the substitute at a point x . Then, $J_f(x)[o(x)]$ is the column corresponding to partial derivatives of the correct label (i.e. the one assigned by the target model). Papernot et al. now add the sign of this column scaled by a factor λ to point x : $x = x + \lambda \text{sign}(J_f(x)[o(x)])$, and add it to their dataset.

By doing this to all current points, they obtain a larger sample set, so the model trained in the next iteration is closer to the target. Note that every epoch, the model is trained from scratch. With enough iterations, the substitute can then be used to generate the adversarial examples.

Ensemble Attacks To improve transferability of adversarial examples created on substitute models, Liu et al. [Liu+16] argue that an adversarial example that works well for multiple models at the same time is bound to transfer better than one that only works on a single one.

That's why they propose that an adversary in the transfer attack setting crafts adversarial examples against an ensemble of models. For a targeted attack, they suggest the negative-log-likelihood-objective:

$$x^* = \arg \min_{x'} - \log\left(\left(\sum_{i=1}^k \alpha_i J_i(x')\right) * \mathbb{1}_{y^*}\right) + \lambda \|x - x'\|$$

where $J(\cdot)$ is each models output, α_i scaling factors for each such that $\sum_i \alpha_i = 1$, for example simply $1/k$.

MI-FGSM and Momentum Kurakin et al. [KGB16] remarked that, while iterative methods yield better results on a single attacked white-box model, single-step methods exhibit better transferability. They suspected that iterative methods overfit to parameters of the model.

To tackle this issue, Dong et al. [Don+18] proposed adding momentum to iterative methods such as iterative FGSM, leading to momentum iterative FGSM. This way, both the underfitting of basic FGSM as well as the overfitting of iterative FGSM is eased.

The gradient is updated as

$$g_{t+1} = \mu * g_t + \frac{\nabla_x J(x_t^*, y)}{\|\nabla_x J(x_t^*, y)\|_1}$$

with μ being the decay factor and the gradient being normalized to handle gradient magnitudes varying over iterations.

Accordingly, the sample is updated as

$$x_{t+1}^* = x_t^* + \alpha * \text{sign}(g_{t+1})$$

α being the step size. For $\mu = 0$, this goes back to being normal FGSM. Note that this change is not restricted to FGSM, but can be applied to any iterative gradient-based method including targeted ones.

ZOO Zeroth-Order Black Box Attacks (ZOO) In 2017, Chen et al. [Che+17a] proposed Zeroth-Order Black Box Attacks (ZOO), a black-box attack without the use of a substitute model. This way, they avoided losses in transferability to the target and achieved comparable results to then-state of the art white box attacks.

This attack is possible if the adversary is allowed to query the target model, i.e. obtaining the confidence scores, $f(x)$ for an arbitrary x . This might be the case for a classification-as-a-service tool. Gradients and model configuration, however, are not available (black box).

To circumvent this restriction, Chen et al. estimate a pseudo-gradient using two queries per coordinate:

$$\hat{g}_i := \frac{\partial f(x)}{\partial x_i} \approx \frac{f(x + he_i) - f(x - he_i)}{2h}$$

where f is a loss function defined on the model's output and h is set to a small constant instead of approaching 0 like in the derivation formula. Naively, this would suffice to estimate full gradients and perform gradient descent. However, this formula requires two evaluations per coordinate, $f(x + he_i)$ and $f(x - he_i)$. Since each pixel translates to three coordinates of input data, this is not a feasible approach.

Instead, Chen et al. perform stochastic coordinate descent, improving just a batch of coordinates at each step. This way, they do not need to estimate the full gradient each time, similar to how stochastic gradient descent only uses a batch of samples at each step. For the optimization itself, they use ADAM or Newton's method.

Still, we need to find a suitable loss function f . In fact, we can use different loss functions to adapt different white-box attacks. In their work, Chen et al. adapted the Carlini-Wagner attack. C-W's loss function performs on the raw logits of a model, an information inaccessible in the black-box case.

Instead, Chen et al. use the logarithm of the confidence scores as a proxy of sorts for the logits and obtain

$$f(x, t) = \max_{i \neq t} \{ \max \{ \log[F(x)]_i \} - \log[F(x)]_t, -\kappa \}$$

for targeted attacks and

$$f(x, t) = \max \{ \log[F(x)]_{t_0} - \max_{i \neq t_0} \{ \log[F(x)]_i \}, -\kappa \}$$

for untargeted ones.

2.4 Notable Poisoning Attacks

Poisoning attacks were already included in [The security of machine learning...], which clustered attacks on machine learning into exploratory and causative, analogous to our evasive and poisoning taxonomy.

The first poisoning attack that got greater attention was Biggio et al.'s gradient-based attack on Support Vector Machines. After, more research was contributed, especially on attacks on neural networks.

Backdoor poisoning attacks With backdoor attacks on machine learning as proposed by Chen et al. [Che+17b] and Gu et al. [GDG17], the adversary's goal is to introduce a backdoor for himself into the model. While the performance on normal inputs is to be kept up, the model should react to specific inputs by misclassifying them. (Either generally, or targeted.) These specific inputs carry a special pattern, the "key" or "trigger".

Gu et al. consider an adversary that has full access to the model or a base version of it. Namely, an ML-as-a-service provider (full access) and a provider of pretrained models (base version access). In both cases, the architecture is specified already, as well as a training set. A backdoor key pattern is chosen by the attacker. The adversary has to find model weights performing well on general samples from the set yet vulnerable to the backdoor.

For that, Gu et al. use training set poisoning, adding a fraction of training images to the dataset again, only with the trigger pattern included and the labels changed according to varying strategies. They did not yet derive a way to optimize poison samples, instead, pattern, label change formula and infected fraction need to be selected by the adversary.

Chen et al. instead use a black-box threat model where the adversary does not have access to model architecture or training data and can only inject a small number of adversary samples including arbitrary labels. The key should not be easily noticeable by a human observer. Crucially, the model performance on non-backdoor inputs should be kept up.

Backdoor keys may either be a single instance or a pattern added to instances. The former ("input-instance-key" strategy) allows injecting just very few poisoned examples while the latter ("pattern-key") can be used to craft adversarial examples from a wide range of inputs, which is often more useful to a hypothetical attack.

In input-instance-key strategies, Chen et al. propose adding different noised versions of the key-to-be to the training set, each with the chosen label. The idea is to get the model to overfit to these.

In Pattern-key strategies, the adversary can blend a key pattern onto pictures, add an accessory, or blend in an accessory according to Chen et al.'s suggestions. The idea of the blending is to make the pattern harder to spot for humans. In any case, the perturbed sample is added to the dataset as poison.

Like Gu et al.'s attack, theirs also depends on hyperparameter choices only, with the framework not optimizing poisoning on its own.

2.5 Clean-Label attacks

Notably, while the backdoor poisoning attack described by Chen et al. does not need access to the model, the adversary has to be able assign arbitrary labels to samples, as does the one by Gu et al. However, this can be highly noticeable and alert human overseers to the attack or might just not be feasible in another way. Clean-label attacks do not give such power to the adversary.

One such attack was proposed by Shafahi et al. [Sha+18]. In contrast to Chen et al.'s attack, the adversary does know the model parameters, which might happen if the target model is based on a pretrained, freely accessible one. His goal is to have a target instance misclassified without modifying it, e.g. getting a malware script to bypass a ML antivirus.

First, he chooses a base instance, such that its class, the "base class", is the one he wants the target instance to be classified as. With this, he crafts a poison instance p by solving

$$p = \arg \min_x ||f(x) - f(t)||_2^2 + \beta ||x - b||_2^2$$

where the first term compares the target t and poison candidates x in feature space (the more similar, the better), and the second penalizes difference from the base instance. The general idea is to find a poison instance close to b , therefore likely to be labeled the same, yet close to t in feature space (output of second-to-last layer), so t is treated the same as b by the model. For optimization, two steps are iterated: First, a gradient step ("forward step"), then a step towards b ("backwards step").

Clean-label backdoor attacks Saha et al. [SSP20] made an effort to combine clean-label attacks with backdoor poisoning attacks, albeit Turner et al. did the same a year earlier [TTM19]. Both also considered how to hide the trigger during poisoning to decrease the noticeability of the backdoor, though differently. Turner et al. manually select a "trigger amplitude", essentially an opaqueness. Saha et al., on the other hand, directly incorporate it into their optimization and not even have the pattern itself in the poisoned instance.

Saha et al. consider a threat model in which the adversary can add poison to a dataset the victim uses for fine-tuning a pretrained model, which the adversary has access to. The scenario is similar to the one described by Gu et al., with the difference that the adversary attacks the fine-tuning, not the pretraining.

The attack itself is similar to Shafani et al.'s. The adversary chooses a source and a target image from the data distribution. He then applies his trigger to the source, obtaining the patched source. Like before, the poison sample is optimised to be close to the target in pixel space (so it probably receives the target label) while close to the patched source in feature space (so the model has difficulties distinguishing them). The notation is flipped, though...

Let $\tilde{s}$ be the patched source, t the target, and ϵ a small, chosen threshold. Saha et al. optimize

$$\arg \min_x \|f(x) - f(\tilde{s})\|_2^2 \quad \text{s.t.} \quad \|f(x) - f(t)\|_\infty^2 < \epsilon$$

to obtain the poisoned sample x . Again, f is the output of the second-to-last layer, the feature space representation. To optimize, they use projected gradient descent (2.3). However, in each optimization step, a new randomly-patched source is selected so the backdoor generalizes to arbitrary images and patch placements.

A more recent work by Zeng et al. [Zen+23] addresses issues with prior work on clean-label backdoor attacks. They consider a black-box threat model where the adversary can only poison a small subset of the training data. The adversary chooses the target class and knows some examples in it. He also has general information on the task the victim model should perform, such as "face recognition", but not specifics like classes used. This enables the attacker to draw up more data instances related to the task, though they might not belong to the original data distribution and are therefore referred to as "public out of distribution (POOD) examples". This is especially easy in image or language tasks where lots of samples are available in the web.

All in all, the adversary is considerably weaker than in aforementioned attacks.

Their approach revolves around optimizing the trigger pattern used under a constraint so it is harder to notice for humans (as in evasion attacks). Suppose the attacker had access to the model the victim would obtain without poisoning. We call this model the oracle f_{orc} . Recall that he also has access to a dataset of the target class D_t . Zeng et al.'s approach is to choose a trigger δ^* maximizing the oracle's confidence on the target class, i.e. solving

$$\delta^* = \arg \min_{\delta \in \Delta} \sum_{(x,t) \in D_t} L(f_{orc}(x + \delta), t)$$

where Δ is the set of allowed perturbations, e.g. the L_∞ -ball. This way, δ^* itself is related to the target class, and adding it to any input increases the likelihood of that input being classified as the target class.

However, the adversary does not have access to f_{orc} . Instead, Zeng et al. propose training a surrogate model f_{sur} on the POOD data D_{POOD} . The above equation becomes

$$\delta^* = \arg \min_{\delta \in \Delta} \sum_{(x,t) \in D_t} L(f_{sur}(x + \delta), t)$$

Now, the attack works as follows. First, f_{sur} is trained using D_{POOD} and D_t . While training the surrogate model on both D_{POOD} and D_t simultaneously would be possible, Zeng et al. suggest

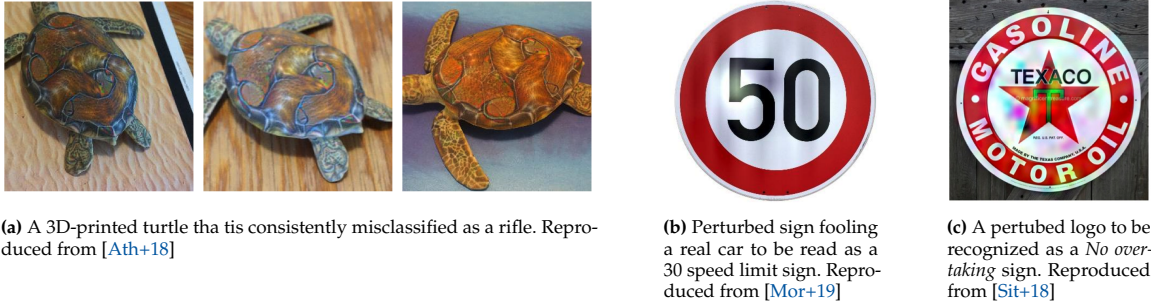


Figure 5: Many creative approaches for transferring adversarial examples into the physical world have been proposed in academic writing; three of which we are showing here. As machine learning application experience increasingly widespread adoption in the real world, many researchers warn of points of attack opening up and possibly enabling severe damage.

training on D_{POOD} first and fine-tune on D_t to make modification for a possible second attack on another class more efficient.

In the second step, they use (stochastic) projected gradient descent to find the trigger δ^* and subsequently add it to a portion of the target class samples D_t . The victim is then given this poisoned target class data set.

In an eventual exploit, the adversary adds the trigger to the desired input. It is an option to amplify the trigger to increase attack success rates.

3 Discussion

3.1 Real-World and Physical Adversarial Attacks

Somewhat surprisingly, in practice, confirmed *malicious* adversarial-example attacks against real institutions or individuals remain strikingly rare.[BR18; RBF23] Two explanations seem possible.

1. Adversarial attacks slip through unnoticed: a successful adversarial input may be logged as an ordinary misclassification, be considered a generic model error, or filed under an unrelated incident category, so that successfully employed adversarial attacks fly under the radar.
2. Adversarial attacks might in practice simply be of little use to actual attackers. Crafting them reliably demands specialised machine-learning and security expertise, whereas cruder techniques—distributed denial-of-service, character obfuscation to slip past spam filters, or merely packing a malware binary—remain cheap, fast, and effective.[Apr+23; RBF23]

The second explanation is the one that we consider to be more plausible. Building an adversarial example to defeat a system that a simpler attack would already defeat is like learning to pick a lock when the crowbar is cheaper, quicker, and works just as well. Interestingly, the few confirmed real-world cases, such as commercial CAPTCHA-solving services or the prompt injection of deployed chatbots, tend to be those where no such simpler alternative exists.

This does not render the phenomenon academic in the pejorative sense. As machine learning is embedded ever more deeply into the physical world—autonomous vehicles, automated border control, biometric surveillance—the question of whether an input that is semantically unchanged to a human can nonetheless fool a sensor pipeline becomes a central engineering concern. The body of work on *physical* adversarial attacks shows that it can: inconspicuous stickers on a stop sign reliably cause targeted misclassification, both in the laboratory and from a moving vehicle;[Ey+18] printable adversarial patches force a chosen label onto almost any scene they appear in;[Bro+17] adversarial eyeglass frames defeat face recognition;[Sha+16] and spoofed sensor returns or crafted road-surface markings can mislead the perception stack of a production self-driving system.[Cao+19; Ten19]

Such results matter because robustness is a core requirement for any safety-relevant deployment—which makes it all the more pressing to understand *why* such systems fail, and what that failure implies. It is to those questions that we are discussing more closely in the following section.

3.2 Explaining Adversarial Examples

Having seen *where* adversarial examples arise, we now turn to *why* they exist, highlighting some influential explanation attempts that have been proposed since their discovery in 2013.

Mysterious Nonlinearity The initial paper by Szegedy et al. that introduced the concept of adversarial attacks, already made various core observations that shaped research ever since. [Sze+13] Firstly, they postulated that adversarial examples are neither random artefacts nor isolated anomalies: they occupy entire regions of the input space—*pockets* in the data manifold—that are sparse, and therefore hard to encounter through random sampling, yet can be produced reliably through optimisation. Second, the effect is not merely overfitting. The same adversarial input often fools different models with different hyperparameters, and even different architectures—the phenomenon later called *transferability*—which already argued against attributing adversarial examples to a single model overfitting its training set.

The missing piece that was not offered by their paper was an explanation of the curious phenomenon. Szegedy et al. do postulate that *highly nonlinear model behaviour* might be behind this issue, but no comprehensive explanation was offered. It was only the next year that such explanation attempt was made.

High-Dimensionality The explanation by Goodfellow et al. [GSS14] can be regarded as one of the most influential in the field. It overturns the intuition, inherited from [Sze+13], that adversarial examples stem from the *extreme non-linearity* of deep networks—highly contorted decision surfaces with rare, low-probability blind spots—possibly compounded by overfitting. Goodfellow et al. argued: linear behaviour in high-dimensional spaces is already *sufficient* to produce adversarial examples; refuting the nonlinearity explanation attempt. Because modern networks are often times deliberately built to locally behave in near-linear ways (eg. using ReLu) for ease of optimisation, the argument carries over from strictly linear models to deep networks.

The mechanism is a high-dimensional *bloating* effect. For a linear model with weight vector w , however, the response to a perturbed input $x + \eta$ —the original input x plus a perturbation η —is $w^\top(x + \eta) = w^\top x + w^\top \eta$, so the perturbation shifts the activation by $w^\top \eta$. The overall output change of a perturbation thus scales linearly with the number of input dimensions n and mean weight magnitude m with respect to the per-pixel change:

$$w^\top \eta = \epsilon \sum_{i=1}^n |w_i| = \epsilon m n.$$

This means that in sufficiently high dimensions, many individually imperceptible, weight-aligned changes accumulate into one large change in the output. Figure 6 illustrates this: the per-pixel budget stays flat as n grows, while the change in the model’s output climbs until it crosses the threshold that flips the prediction.

This point is easily overstated, and often is: Goodfellow et al. do not claim that linearity is a necessary ingredient of the explanation. They show only that non-linearity is not necessary—the simplest possible model, plain logistic or softmax regression with no hidden layer, already exhibits adversarial examples—as they showcased with FGSM which we discussed in section 2.3. (The refined, later view holds that both dimensionality and local curvature matter, as reflected in the stronger iterative and minimum-norm attacks discussed above. [Mad+17; MFF16])

The linear view also provided the first explanation of transferability. Since adversarial examples now occupy broad, contiguous half-spaces rather than fine pockets, different models’ adversarial regions overlap heavily, which explains why an example crafted for one model tends to fool another at all. And because the discriminative direction is largely a property of the data rather than of the model, models trained on the same task tend to learn similar weight vectors, so that the same perturbation pushes them toward the same incorrect class—which explains why the models tend to agree on that class.

The Fundamental Mismatch Ilyas et al. [Ily+19] shift the framing once more, reframing adversarial examples not as pathological failures but as a *mismatch between machine and human perception*: models

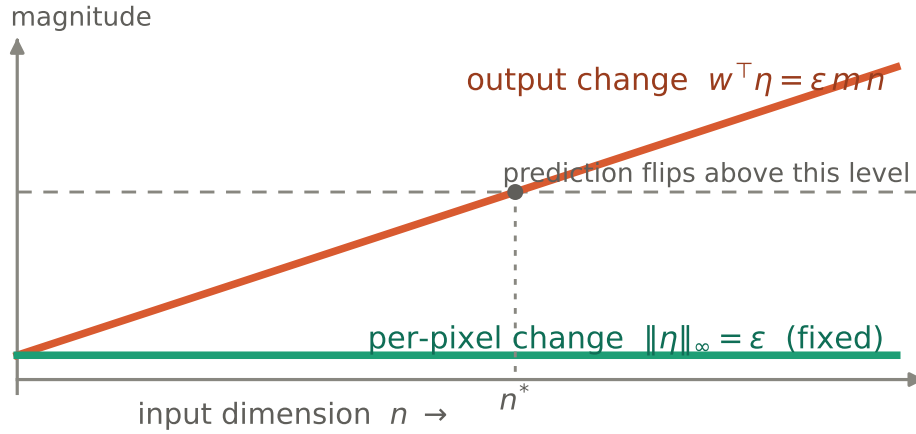


Figure 6: The linear explanation of adversarial examples. Under an L_∞ (per-pixel) budget every input coordinate is perturbed by at most ϵ , so the perceptual size of the perturbation, $\|\eta\|_\infty = \epsilon$, stays fixed as the input dimension n grows. The induced change in the model’s activation, $w^\top \eta = \epsilon m n$, instead grows *linearly* with n . Once the dimension is large enough (at n^*) this accumulated change crosses the threshold that flips the prediction—so in high dimensions an imperceptible perturbation suffices.

exploit patterns that are genuinely predictive, but imperceptible to, and unintuitive for, humans. They distinguish *predictive* features (features correlated with the label) from *robust* features (features that remain predictive under perturbation). Since *empirical risk minimisation* rewards any signal that improves accuracy, a learner will readily rely on predictive but non-robust features; adversarial vulnerability is then an expected consequence of standard training, rooted in the gap between the human sense of similarity and the statistical structure of the data. A striking experiment makes this concrete: they relabel every image of CIFAR-10 with a new, random target class and perturb it so that a model reads the new label, while to a human the images appear unchanged and hence entirely mislabelled. A standard model trained on this dataset nevertheless generalises well to the *clean*, unmodified test set—evidence that it has latched onto real, transferable features that merely happen to be imperceptible to us. This view also bears on interpretability: if a model succeeds by relying on human-imperceptible features, there may be little in its decisions for a human to understand.

3.3 Pertinence of Adversarial Examples as a Subject Area

The two preceding subsections shed an interesting light on adversarial attacks as a field of study in the domain of cyber security. In the first, we asked whether they pose a real danger and found that little evidence can be found that adversarial attacks pose a pertinent security threat in any practical scenario. As a live security threat they have so far been marginal, dwarfed by cruder and cheaper forms of attack. [Apr+23; RBF23] Adding in our examination of different explanations for the phenomenon, we come to the conclusion that the primary significance of adversarial examples is of *epistemic* nature. Their main impact can be seen as offering a new view on the inherent vulnerabilities of machine learning systems; shaking up pre-existing beliefs. In other words, their importance lies less in what they let an attacker do than in what they reveal about the systems they target.

What they reveal is considerable. Before the discovery and systematic study of adversarial examples, machine-learning models were evaluated almost exclusively on accuracy, and robustness was expected only implicitly. [BR18; HD19] Adversarial examples showed that high test accuracy does not imply that a model has grasped a learning task the way a human would: a classifier will confidently mislabel an input that is, to a human observer, semantically unchanged. This was the original observation that made the phenomenon of adversarial attacks so remarkable. This forced a more intricate account of how machine learners operate—from Goodfellow et al.’s high-dimensional linearity to Ilyas et al.’s reframing of adversarial features as genuine but human-imperceptible signal [GSS14; Ily+19]—and, with it, a redefinition of *robustness* from an assumed by-product of accuracy into a distinct property that must be explicitly demanded, engineered, and evaluated. This

shift is not merely academic: as machine learning is embedded in autonomous vehicles, border control, and surveillance, physical adversarial attacks make plain that robustness cannot be taken for granted in exactly those deployments where its absence would be most costly.

Viewed from a distance, the lesson generalises well beyond the field. When we trust a simple mathematical proxy to stand in for a messy real-world objective—as an overriding focus on accuracy does for machine learners—we are bound to be disappointed, and the resulting mismatch can surface suddenly and in unexpected ways. The contribution of adversarial examples is to have made that mismatch—between human and machine notions of similarity and understanding—impossible to ignore.

Statement of Contributions

Please indicate here the contributions of each single author/student. This section can be dropped if there is a single author only.

References

- [BRB17] Wieland Brendel, Jonas Rauber, and Matthias Bethge. “Decision-based adversarial attacks: Reliable attacks against black-box machine learning models”. In: *arXiv preprint arXiv:1712.04248* (2017).
- [Sze+13] Christian Szegedy et al. “Intriguing properties of neural networks”. In: *arXiv preprint arXiv:1312.6199* (2013).
- [GSS14] Ian J Goodfellow, Jonathon Shlens, and Christian Szegedy. “Explaining and harnessing adversarial examples”. In: *arXiv preprint arXiv:1412.6572* (2014).
- [Mad+17] Aleksander Madry et al. “Towards deep learning models resistant to adversarial attacks”. In: *arXiv preprint arXiv:1706.06083* (2017).
- [Pit+19] Nikolaos Pitropakis et al. “A taxonomy and survey of attacks against machine learning”. In: *Computer Science Review* 34 (2019), p. 100199.
- [Pap+16] Nicolas Papernot et al. “Distillation as a defense to adversarial perturbations against deep neural networks”. In: *2016 IEEE symposium on security and privacy (SP)*. IEEE, 2016, pp. 582–597.
- [MFF16] Seyed-Mohsen Moosavi-Dezfooli, Alhussein Fawzi, and Pascal Frossard. “Deepfool: a simple and accurate method to fool deep neural networks”. In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2016, pp. 2574–2582.
- [Cha+18] Anirban Chakraborty et al. “Adversarial attacks and defences: A survey”. In: *arXiv preprint arXiv:1810.00069* (2018).
- [Cos+24] Joana C Costa et al. “How deep learning sees the world: A survey on adversarial attacks & defenses”. In: *IEEE Access* 12 (2024), pp. 61113–61136.
- [Zha+18] Richard Zhang et al. “The unreasonable effectiveness of deep features as a perceptual metric”. In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2018, pp. 586–595.
- [Lia+22] Hongshuo Liang et al. “Adversarial attack and defense: A survey”. In: *Electronics* 11.8 (2022), p. 1283.
- [PMG16] Nicolas Papernot, Patrick McDaniel, and Ian Goodfellow. “Transferability in machine learning: from phenomena to black-box attacks using adversarial samples”. In: *arXiv preprint arXiv:1605.07277* (2016).
- [CW17] Nicholas Carlini and David Wagner. “Towards evaluating the robustness of neural networks”. In: *2017 IEEE symposium on security and privacy (SP)*. IEEE, 2017, pp. 39–57.
- [KGB17a] Alexey Kurakin, Ian J. Goodfellow, and Samy Bengio. “Adversarial examples in the physical world”. In: *CoRR abs/1607.02533* (2017). arXiv: 1607.02533. URL: <http://arxiv.org/abs/1607.02533>.
- [Che+17a] Pin-Yu Chen et al. “ZOO: Zeroth Order Optimization Based Black-box Attacks to Deep Neural Networks without Training Substitute Models”. In: *Proceedings of the 10th ACM Workshop on Artificial Intelligence and Security*. CCS ’17. ACM, Nov. 2017, pp. 15–26. DOI: 10.1145/3128572.3140448. URL: <http://dx.doi.org/10.1145/3128572.3140448>.

- 642 [GDG17] Tianyu Gu, Brendan Dolan-Gavitt, and Siddharth Garg. “Badnets: Identifying vulnera-
643 bilities in the machine learning model supply chain”. In: *arXiv preprint arXiv:1708.06733*
644 (2017).
- 645 [Che+17b] Xinyun Chen et al. “Targeted backdoor attacks on deep learning systems using data
646 poisoning”. In: *arXiv preprint arXiv:1712.05526* (2017).
- 647 [Sha+18] Ali Shafahi et al. “Poison frogs! targeted clean-label poisoning attacks on neural net-
648 works”. In: *Advances in neural information processing systems* 31 (2018).
- 649 [TTM19] Alexander Turner, Dimitris Tsipras, and Aleksander Madry. “Label-consistent backdoor
650 attacks”. In: *arXiv preprint arXiv:1912.02771* (2019).
- 651 [SSP20] Aniruddha Saha, Akshayvarun Subramanya, and Hamed Pirsiavash. “Hidden trigger
652 backdoor attacks”. In: *Proceedings of the AAAI conference on artificial intelligence*. Vol. 34.
653 07. 2020, pp. 11957–11965.
- 654 [Zen+23] Yi Zeng et al. “Narcissus: A practical clean-label backdoor attack with limited informa-
655 tion”. In: *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communica-*
656 *tions Security*. 2023, pp. 771–785.
- 657 [KGB17b] Alexey Kurakin, Ian J. Goodfellow, and Samy Bengio. “Adversarial Machine Learning
658 at Scale”. In: *CoRR abs/1611.01236* (2017). arXiv: 1611.01236. URL: <http://arxiv.org/abs/1611.01236>.
- 659 [KB14] Diederik P Kingma and Jimmy Ba. “Adam: A method for stochastic optimization”. In:
660 *arXiv preprint arXiv:1412.6980* (2014).
- 661 [Pap+17] Nicolas Papernot et al. “Practical black-box attacks against machine learning”. In:
662 *Proceedings of the 2017 ACM on Asia conference on computer and communications security*.
663 2017, pp. 506–519.
- 664 [Big+13] Battista Biggio et al. “Evasion attacks against machine learning at test time”. In: *Joint*
665 *European conference on machine learning and knowledge discovery in databases*. Springer.
666 2013, pp. 387–402.
- 667 [Liu+16] Yanpei Liu et al. “Delving into transferable adversarial examples and black-box attacks”.
668 In: *arXiv preprint arXiv:1611.02770* (2016).
- 669 [KGB16] Alexey Kurakin, Ian Goodfellow, and Samy Bengio. “Adversarial machine learning at
670 scale”. In: *arXiv preprint arXiv:1611.01236* (2016).
- 671 [Don+18] Yinpeng Dong et al. “Boosting adversarial attacks with momentum”. In: *Proceedings of*
672 *the IEEE conference on computer vision and pattern recognition*. 2018, pp. 9185–9193.
- 673 [BR18] Battista Biggio and Fabio Roli. “Wild patterns: Ten years after the rise of adversarial
674 machine learning”. In: *Pattern Recognition* 84 (2018), pp. 317–331.
- 675 [RBF23] Edward Raff, Michel Benaroch, and Andrew L. Farris. “You Don’t Need Robust Ma-
676 chine Learning to Manage Adversarial Attack Risks”. In: *arXiv preprint arXiv:2306.09951*
677 (2023).
- 678 [Apr+23] Giovanni Apruzzese et al. ““Real Attackers Don’t Compute Gradients”: Bridging the
679 Gap Between Adversarial ML Research and Practice”. In: *2023 IEEE Conference on Secure*
680 *and Trustworthy Machine Learning (SaTML)*. IEEE. 2023, pp. 339–364.
- 681 [Eyk+18] Kevin Eykholt et al. “Robust Physical-World Attacks on Deep Learning Visual Classifi-
682 cation”. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*
683 (CVPR). 2018, pp. 1625–1634.
- 684 [Bro+17] Tom B. Brown et al. “Adversarial Patch”. In: *NIPS 2017 Workshop on Machine Learning*
685 *and Computer Security*. arXiv:1712.09665. 2017.
- 686 [Sha+16] Mahmood Sharif et al. “Accessorize to a Crime: Real and Stealthy Attacks on State-
687 of-the-Art Face Recognition”. In: *Proceedings of the 2016 ACM SIGSAC Conference on*
688 *Computer and Communications Security (CCS)*. 2016, pp. 1528–1540.
- 689 [Cao+19] Yulong Cao et al. “Adversarial Sensor Attack on LiDAR-based Perception in Au-
690 tonomous Driving”. In: *Proceedings of the 2019 ACM SIGSAC Conference on Computer*
691 *and Communications Security (CCS)*. 2019, pp. 2267–2281.
- 692 [Ten19] Tencent Keen Security Lab. *Experimental Security Research of Tesla Autopilot*. <https://keenlab.tencent.com/en/2019/03/29/Tencent-Keen-Security-Lab-Experimental-Security-Research-of-Tesla-Autopilot/>. 2019.
- 693 [Ath+18] Anish Athalye et al. “Synthesizing robust adversarial examples”. In: *International*
694 *conference on machine learning*. PMLR. 2018, pp. 284–293.

- 698 [Mor+19] Nir Morgulis et al. "Fooling a real car with adversarial traffic signs". In: *arXiv preprint*
699 *arXiv:1907.00374* (2019).
- 700 [Sit+18] Chawin Sitawarin et al. "Darts: Deceiving autonomous cars with toxic signs". In: *arXiv*
701 *preprint arXiv:1802.06430* (2018).
- 702 [Ily+19] Andrew Ilyas et al. "Adversarial examples are not bugs, they are features". In: *Advances*
703 *in Neural Information Processing Systems* 32 (2019).
- 704 [HD19] Dan Hendrycks and Thomas Dietterich. "Benchmarking neural network robustness
705 to common corruptions and perturbations". In: *International Conference on Learning*
706 *Representations (ICLR)*. 2019.

Zentrales Prüfungsamt/Central Examination Office



Eidesstattliche Versicherung

Statutory Declaration in Lieu of an Oath

Name, Vorname/Last Name, First Name

Matrikelnummer (freiwillige Angabe)

Matriculation No. (optional)

Ich versichere hiermit an Eides Statt, dass ich die vorliegende Arbeit/Bachelorarbeit/
Masterarbeit* mit dem Titel

I hereby declare in lieu of an oath that I have completed the present paper/Bachelor thesis/Master thesis* entitled

selbstständig und ohne unzulässige fremde Hilfe (insbes. akademisches Ghostwriting)
erbracht habe. Ich habe keine anderen als die angegebenen Quellen und Hilfsmittel benutzt.
Für den Fall, dass die Arbeit zusätzlich auf einem Datenträger eingereicht wird, erkläre ich,
dass die schriftliche und die elektronische Form vollständig übereinstimmen. Die Arbeit hat in
gleicher oder ähnlicher Form noch keiner Prüfungsbehörde vorgelegen.

independently and without illegitimate assistance from third parties (such as academic ghostwriters). I have used no other than
the specified sources and aids. In case that the thesis is additionally submitted in an electronic format, I declare that the written
and electronic versions are fully identical. The thesis has not been submitted to any examination body in this, or similar, form.

Ort, Datum/City, Date

Unterschrift/Signature

*Nichtzutreffendes bitte streichen

*Please delete as appropriate

Belehrung:**Official Notification:****§ 156 StGB: Falsche Versicherung an Eides Statt**

Wer vor einer zur Abnahme einer Versicherung an Eides Statt zuständigen Behörde eine solche Versicherung
falsch abgibt oder unter Berufung auf eine solche Versicherung falsch aussagt, wird mit Freiheitsstrafe bis zu drei
Jahren oder mit Geldstrafe bestraft.

Para. 156 StGB (German Criminal Code): False Statutory Declarations

Whoever before a public authority competent to administer statutory declarations falsely makes such a declaration or falsely
testifies while referring to such a declaration shall be liable to imprisonment not exceeding three years or a fine.

§ 161 StGB: Fahrlässiger Falscheid; fahrlässige falsche Versicherung an Eides Statt

(1) Wenn eine der in den §§ 154 bis 156 bezeichneten Handlungen aus Fahrlässigkeit begangen worden ist, so
tritt Freiheitsstrafe bis zu einem Jahr oder Geldstrafe ein.

(2) Strafflosigkeit tritt ein, wenn der Täter die falsche Angabe rechtzeitig berichtet. Die Vorschriften des § 158
Abs. 2 und 3 gelten entsprechend.

Para. 161 StGB (German Criminal Code): False Statutory Declarations Due to Negligence

(1) If a person commits one of the offences listed in sections 154 through 156 negligently the penalty shall be imprisonment not
exceeding one year or a fine.

(2) The offender shall be exempt from liability if he or she corrects their false testimony in time. The provisions of section 158 (2)
and (3) shall apply accordingly.

Die vorstehende Belehrung habe ich zur Kenntnis genommen:

I have read and understood the above official notification:

Ort, Datum/City, Date

Unterschrift/Signature